

Chapter 42

Recommender Systems & Ranking

Build a popularity baseline, and know why it's hard to beat with little data · build a content-based recommender from item descriptions, reusing Chapter 41's TF-IDF and cosine similarity · build item-based collaborative filtering by hand and in scikit-learn

Approved chapter · v1

Contents

Chapter 42. Recommender Systems & Ranking

Why this matters

In plain English

42.1 The data, and a popularity baseline

42.2 Item-based collaborative filtering

42.3 Evaluating recommendations: precision@k and NDCG@k

42.4 Matrix factorization and implicit feedback

42.5 Content-based filtering

42.6 The cold-start problem

42.7 B2B cross-selling and ranking by segment

Common mistakes and how to spot them

In the real world: the recommendation that didn't need a model

Tools

The project: "customers who bought this also bought" for Riverstone

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 42. Recommender Systems & Ranking

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: build a popularity baseline, and know why it's hard to beat with little data · build a content-based recommender from item descriptions, reusing Chapter 41's TF-IDF and cosine similarity · build item-based collaborative filtering by hand and in scikit-learn · understand matrix factorization and implicit feedback, and fit both with scikit-learn and the `implicit` library · evaluate recommendations with `precision@k` and `NDCG@k`, not accuracy · handle the cold-start problem for new products and new customers · combine methods into a hybrid recommender · apply all of this to B2B cross-selling.

Before you start: Chapter 35 (cosine similarity, dot products), Chapter 37 (accounts data), Chapter 38 (the basket data and market basket rules), Chapter 41 (TF-IDF). This chapter turns those pieces into personalized recommendations.

Time needed: 6–9 hours over one week.

Tools: Python 3 with scikit-learn, plus the `implicit` library (free).

Practice data: Riverstone's order-line data from Chapter 38 (33,931 orders, 4,516 accounts, 24 products) and the accounts data from Chapter 37, with a short product description added to each catalog item for this chapter. Every number in this chapter was calculated, and every output shown is real.

Why this matters

Chapter 38's market basket rules answered one question well: *which products tend to appear in the same order?* That's useful for a "frequently bought together" prompt on a product page, but it's the same answer for every customer. A recommender system answers a sharper question: *what should we suggest to this specific customer, right now?*

- Riverstone's account managers want a short list per customer: "these three products this account hasn't bought yet, that accounts like them buy a lot."
- A new product launches with zero sales history. Nobody has bought it yet, so nothing knows to recommend it — except its description.
- A brand-new account places its first order. There's no purchase history to learn from at all.

Those three situations need three different tools, and knowing which to reach for, and how to tell whether any of them actually works, is what this chapter teaches.

In plain English

Think of an experienced Riverstone sales rep who's worked every territory for ten years.

Ask what to suggest to a customer who's never ordered before, and the rep falls back on "well, everyone buys food container lids" — that's a **popularity baseline**. Ask about a customer who buys garden furniture, and the rep says "show them the matching cushions, they're clearly in the same range" — that's **content-based filtering**, reasoning from what the product is. Ask about a customer who buys a lot like another customer three towns over, and the rep says "that other account also picked up dolly wheels after their pallet boxes, worth a try" — that's **collaborative filtering**, reasoning from what *similar customers* did, with no idea what a dolly wheel actually looks like.

A rep who's really good does both at once, weighing them: mostly go with what similar customers bought, but lean on the product description when a customer or a product is too new to have much history. That blend is a **hybrid recommender**, and knowing when the rep's gut feeling is actually working, rather than just sounding confident, is **evaluation**.

42.1 The data, and a popularity baseline

```
import sys
import warnings

import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")
sys.path.append(".")
from products_text import load_products

lines = pd.read_csv("../baskets/order_lines.csv", parse_dates=["order_date"])
products = load_products()
accounts = pd.read_csv("../accounts/accounts.csv")
lines = lines.merge(
    products[["product_id", "product_name", "category"]], on="product_id"
)
lines = lines.merge(accounts[["account_id", "segment"]], on="account_id")
print(
    f"{len(lines):,} order lines, {lines['order_id'].nunique():,} orders, "
    f"{lines['account_id'].nunique():,} accounts, {lines['product_id'].nunique():,} products"
)

interactions = (
    lines.groupby(["account_id", "product_id"])["quantity"].sum().reset_index()
)
matrix = interactions.pivot(
    index="account_id", columns="product_id", values="quantity"
).fillna(0)
print(f"account x product matrix: {matrix.shape[0]:,} rows, {matrix.shape[1]} columns")
print(f"density (share of cells that are non-zero): {(matrix > 0).mean().mean():.1%}")
```

```
92,359 order lines, 33,931 orders, 4,516 accounts, 24 products
account x product matrix: 4,516 rows, 24 columns
density (share of cells that are non-zero): 38.4%
```

Riverstone's 24-product catalog is deliberately small for this chapter, which makes every number easy to check by hand; real retail catalogs run to thousands or millions of items, and Chapter 48 covers the engineering that scale demands. The account-by-product **interaction matrix** here is 38% full, unusually dense for a recommender problem — most catalogs are well under 1% dense, since no customer buys more than a sliver of what's on offer. Riverstone's density comes from having only 24 products; hold onto that fact, because a couple of results below only make sense in light of it.

The simplest possible recommender needs no personalization at all:

```
popularity = (matrix > 0).sum(axis=0).sort_values(ascending=False)
popularity_share = popularity / matrix.shape[0]
print("most popular products (share of accounts that bought them):")
print((popularity_share.head(5) * 100).round(1).astype(str).add("%").to_string())
print("\nleast popular:")
print((popularity_share.tail(3) * 100).round(1).astype(str).add("%").to_string())
```

```
most popular products (share of accounts that bought them):
```

product_id	
P09	64.9%
P03	57.0%
P04	54.7%
P06	50.8%
P08	47.1%

```
least popular:
```

product_id	
P17	23.6%
P15	23.5%
P13	23.5%

Reading it. The Airtight Seal Pack is bought by 65% of accounts, the least popular industrial items by under a quarter. **Popularity is the seasonal-naive of recommenders** (Chapter 40): cheap, needs no history, and a real baseline every fancier method must beat. It recommends the same thing to everyone, which is exactly its weakness — a wholesale drum buyer gets pointed at garden furniture accessories just because they're popular overall.

42.2 Item-based collaborative filtering

The idea, by hand

Collaborative filtering finds patterns in *who bought what*, with no knowledge of what the products are. **Item-based** filtering asks: which products are bought by similar sets of customers? Two products are similar if the customers who buy one tend to also buy the other, measured with cosine similarity on the columns of the interaction matrix — exactly Chapter 35's cosine similarity, now comparing products by their customers instead of customers by their products.

```

toy = pd.DataFrame(
    {
        "Storage Crate": [3, 4, 0, 0, 8],
        "Food Container": [0, 0, 6, 5, 0],
        "Crate Lid": [5, 3, 0, 0, 10],
        "Chopping Board": [0, 0, 2, 3, 1],
    },
    index=[
        "Sharma Hardware",
        "Metro Mart",
        "Coastal Foods",
        "Royal Banquets",
        "Prime Wholesale",
    ],
)
print(toy)

crate = toy["Storage Crate"].to_numpy()
lid = toy["Crate Lid"].to_numpy()
container = toy["Food Container"].to_numpy()
cos_crate_lid = (crate @ lid) / (np.linalg.norm(crate) * np.linalg.norm(lid))
cos_crate_container = (crate @ container) / (
    np.linalg.norm(crate) * np.linalg.norm(container)
)
print(f"\ncosine(Storage Crate, Crate Lid) = {cos_crate_lid:.3f}")
print(f"cosine(Storage Crate, Food Container) = {cos_crate_container:.3f}")

```

	Storage Crate	Food Container	Crate Lid	Chopping Board
Sharma Hardware	3	0	5	0
Metro Mart	4	0	3	0
Coastal Foods	0	6	0	2
Royal Banquets	0	5	0	3
Prime Wholesale	8	0	10	1

cosine(Storage Crate, Crate Lid) = 0.980
 cosine(Storage Crate, Food Container) = 0.000

Reading it. Storage Crate and Crate Lid are bought together so consistently (every customer who buys one buys the other, in roughly matching quantities) that their cosine similarity is 0.980, nearly the maximum of 1. Storage Crate and Food Container share no customers at all in this toy example, so their similarity is exactly 0: two products can be popular individually and still have nothing in common, if no one buys both.

The full matrix

```

from sklearn.metrics.pairwise import cosine_similarity

X = matrix.to_numpy()
items = matrix.columns.tolist()
item_similarity = cosine_similarity(X.T)
np.fill_diagonal(item_similarity, 0) # a product is not its own recommendation
sim_table = pd.DataFrame(item_similarity, index=items, columns=items)

target = "P01" # Storage Crate 50L
top_similar = sim_table[target].sort_values(ascending=False).head(4)
names = products.set_index("product_id")["product_name"]
print(f"products bought most like {names[target]!r}:")
for product_id, score in top_similar.items():
    print(f" {names[product_id]:<22} similarity {score:.3f}")

```

```
products bought most like 'Storage Crate 50L':  
Crate Lid (50L)          similarity 0.677  
Crate Lid (80L)          similarity 0.587  
Dolly Wheels (set)       similarity 0.561  
Drum Tap Fitting         similarity 0.558
```

Reading it. With no product descriptions involved anywhere in this calculation, the top match for Storage Crate 50L is its own lid, exactly as section 42.1's popularity numbers would predict from Chapter 38's planted co-purchase rule. The second and third matches, Crate Lid (80L) and Dolly Wheels, are products bought by similar *kinds* of customers (frequent, larger-volume buyers) rather than obviously related items. That's the signature of collaborative filtering: sometimes it rediscovers an obvious pairing, and sometimes it surfaces a pairing that makes business sense once you see it, but that no one would have listed by looking at the product alone.

42.3 Evaluating recommendations: precision@k and NDCG@k

Why accuracy doesn't apply

A recommender doesn't predict one right answer; it produces a **ranked list**, and success means a good item appears *somewhere near the top* of that list. Two measures fit that shape:

- **Precision@k**: of the k items recommended, what share does the customer actually want? If a 5-item list contains 1 product the customer goes on to buy, precision@5 is 20%.
- **NDCG@k** (normalized discounted cumulative gain): like precision, but a hit at rank 1 counts more than a hit at rank 5. Each hit's value is discounted by $1 \div \log_2(\text{rank} + 2)$, then the total is divided by the best possible score (a hit at rank 1) to normalize it to between 0 and 1.

The evaluation setup: leave-one-out

To test a recommender honestly, hide one thing each customer actually bought, ask the recommender to guess it back from everything else the customer bought, and check whether it appears in the top k :

```

rng = np.random.default_rng(42)
n_owned = (X > 0).sum(axis=1)
eligible = np.where(n_owned >= 2)[
    0
] # accounts with at least 2 products can have one held out
held_out = np.full(X.shape[0], -1)
for i in eligible:
    owned = np.where(X[i] > 0)[0]
    held_out[i] = rng.choice(owned)

X_train = X.copy()
X_train[eligible, held_out[eligible]] = 0
print(
    f"{len(eligible):,} of {X.shape[0]:,} accounts have 2+ products and can be evaluated"
)
print(
    f"one held-out product removed from each; {X_train.sum():,.0f} units remain "
    f"of {X.sum():,.0f} original"
)

```

```

4,309 of 4,516 accounts have 2+ products and can be evaluated
one held-out product removed from each; 1,482,144 units remain of 1,614,385 original

```

```

def evaluate(score_fn, k=5):
    hits = 0
    ndcg_total = 0.0
    for i in eligible:
        scores = score_fn(i).copy()
        scores[X_train[i] > 0] = -np.inf # never recommend something already bought
        top_k = np.argsort(-scores)[:k]
        if held_out[i] in top_k:
            hits += 1
            rank = int(np.where(top_k == held_out[i])[0][0])
            ndcg_total += 1 / np.log2(rank + 2)
    n = len(eligible)
    return hits / n, ndcg_total / n

pop_counts = (X_train > 0).sum(axis=0).astype(float)
precision_pop, ndcg_pop = evaluate(lambda i: pop_counts)
print(f"popularity baseline: precision@5 {precision_pop:.1%}  NDCG@5 {ndcg_pop:.3f}")

```

```

popularity baseline: precision@5 61.4%  NDCG@5 0.428

```

How it works: for each eligible account, one purchased product is hidden from `X_train`; a good recommender should place that hidden product near the top of what it suggests, having never been told it belonged there. Products the account already owns are excluded from the recommendation list (`scores[X_train[i] > 0] = -np.inf`), since re-recommending something a customer already has isn't useful. This is the same discipline as Chapter 36's train/test split, adapted to ranking.

NDCG by hand

```
relevance = [0, 1, 0, 0, 0] # the held-out item was ranked 2nd (index 1) of 5
dcg = sum(rel / np.log2(idx + 2) for idx, rel in enumerate(relevance))
ideal = 1 / np.log2(1 + 1) # best possible: the relevant item ranked 1st
print(
    f"DCG for this list: {dcg:.3f}    ideal DCG: {ideal:.3f}    NDCG: {dcg / ideal:.3f}"
)
print(
    f"a hit at rank 1 scores {1 / np.log2(2):.3f}; "
    f"the same hit at rank 5 scores only {1 / np.log2(6):.3f}"
)
```

```
DCG for this list: 0.631    ideal DCG: 1.000    NDCG: 0.631
a hit at rank 1 scores 1.000; the same hit at rank 5 scores only 0.387
```

Reading it. A hit at rank 1 is worth exactly 1.0; the same hit pushed to rank 5 is worth only 0.387, a 61% discount, because rank 5 is much less useful to act on than rank 1 (a rep with time for one call wants the top suggestion to be right). This is why NDCG, not precision, is the standard reported metric in the recommender literature: it rewards getting the *order* right, not just the presence of a good item somewhere in the list.

Reading it. The popularity baseline (block F above) scores 61.4% precision@5 and 0.428 NDCG@5 on this same held-out test. Now item-based collaborative filtering:

```
item_sim_train = cosine_similarity(X_train.T)
np.fill_diagonal(item_sim_train, 0)

def item_cf_score(i):
    return X_train[i] @ item_sim_train

precision_cf, ndcg_cf = evaluate(item_cf_score)
print(
    f"item-based collaborative filtering: precision@5 {precision_cf:.1%}"
    f"    NDCG@5 {ndcg_cf:.3f}"
)
```

```
item-based collaborative filtering: precision@5 67.6%    NDCG@5 0.510
```

Reading it. Item-based collaborative filtering beats popularity clearly (67.6% against 61.4% precision@5; NDCG 0.510 against 0.428): using *who bought what together* genuinely improves on suggesting the same popular items to everyone.

42.4 Matrix factorization and implicit feedback

The idea

Matrix factorization compresses the account-by-product matrix into two smaller matrices — an account-by-factor matrix and a factor-by-product matrix — such that multiplying them back together approximately reconstructs the original. The **factors** are learned, not chosen: they might end up meaning something like "buys a lot of industrial gear" or "buys kitchenware in small quantities," but nobody tells the model what to call them, exactly as PCA's components in Chapter 35 had no names until you looked at their loadings.

```
from sklearn.decomposition import TruncatedSVD

for n_factors in [4, 8, 12]:
    svd = TruncatedSVD(n_components=n_factors, random_state=42).fit(X_train)
    account_factors = svd.transform(X_train)
    reconstructed = account_factors @ svd.components_
    precision_svd, ndcg_svd = evaluate(lambda i, r=reconstructed: r[i])
    variance_kept = svd.explained_variance_ratio_.sum()
    print(
        f"{n_factors:>2} factors: precision@5 {precision_svd:.1%}"
        f"    NDCG@5 {ndcg_svd:.3f}    "
        f"variance kept {variance_kept:.1%}"
    )
```

4 factors:	precision@5 66.2%	NDCG@5 0.493	variance kept 62.2%
8 factors:	precision@5 55.5%	NDCG@5 0.397	variance kept 74.2%
12 factors:	precision@5 49.0%	NDCG@5 0.329	variance kept 83.3%

Reading it, plainly. More factors capture more of the matrix's variance (up to 83% at 12 factors), and the recommendations get **worse**, not better: precision@5 falls from 66.2% to 49.0% as factors rise from 4 to 12. With only 24 products, a handful of factors already explains most of the real structure; extra factors fit noise in a $4,516 \times 24$ matrix that has very little of it to fit. This is Chapter 37's bias-variance trade-off again, in a new setting: more capacity is not automatically better, and the honest way to know is to test it, not to assume that "more factors captures more nuance" is a good thing.

Implicit feedback

Riverstone's data has no five-star ratings, only purchase quantities: **implicit feedback**. A customer buying 40 units of something is a much stronger signal than buying 1, but it isn't a rating — someone who buys 40 crate lids isn't "40 times more satisfied" than someone who buys 1, they just run a bigger business. The standard approach (Hu, Koren & Volinsky's algorithm, implemented in the `implicit` library) treats every purchased quantity as a **confidence level** on a *binary* "did they buy it" signal, rather than as a rating to predict directly, and log-transforms raw counts first, exactly as Chapter 35 log-transformed skewed order values.

```

import implicit
from scipy.sparse import csr_matrix

confidence = csr_matrix(
    np.log1p(X_train)
) # dampen large quantities (Chapter 35's log transform)
for n_factors in [8, 16]:
    model = implicit.als.AlternatingLeastSquares(
        factors=n_factors, regularization=0.1, iterations=20, random_state=42
    )
    model.fit(confidence)

    def als_score(i, m=model):
        return m.user_factors[i] @ m.item_factors.T

    precision_als, ndcg_als = evaluate(als_score)
    print(
        f"ALS, {n_factors}>2} factors: precision@5 {precision_als:.1%}"
        f"    NDCG@5 {ndcg_als:.3f}"
    )

```

```

ALS, 8 factors: precision@5 69.8%    NDCG@5 0.489
ALS, 16 factors: precision@5 46.6%    NDCG@5 0.348

```

Reading it. With 8 factors, `implicit`'s ALS reaches the best result in this chapter: 69.8% precision@5. With 16 factors, it collapses even further than the SVD did (46.6%), the same overfitting pattern, more pronounced. **The lesson generalizes beyond this dataset:** on a small or narrow catalog, keep factor counts low and always check with a held-out test, rather than trusting a bigger, fancier-sounding model by default.

42.5 Content-based filtering

Reusing Chapter 41

Every catalog item has a short text description. Turning those into TF-IDF vectors and comparing them with cosine similarity — precisely Chapter 41's pipeline, unchanged — gives a recommender that needs no purchase history at all:

```

from sklearn.feature_extraction.text import TfidfVectorizer

description_by_item = products.set_index("product_id").loc[items, "description"]
tfidf = TfidfVectorizer(stop_words="english")
description_vectors = tfidf.fit_transform(description_by_item)
content_similarity = cosine_similarity(description_vectors)
print("most similar products to Storage Crate 50L by description text alone:")
content_table = pd.DataFrame(content_similarity, index=items, columns=items)
top_content = content_table["P01"].drop("P01").sort_values(ascending=False).head(4)
for product_id, score in top_content.items():
    print(f" {names[product_id]:<22} similarity {score:.3f}")

def content_score(i):
    return X_train[i] @ content_similarity

precision_content, ndcg_content = evaluate(content_score)
print(
    f"\ncontent-based filtering: precision@5 {precision_content:.1%}"
    f"    NDCG@5 {ndcg_content:.3f}"
)

```

```

most similar products to Storage Crate 50L by description text alone:
Storage Crate 80L      similarity 0.539
Crate Lid (50L)        similarity 0.296
Food Container 5L       similarity 0.249
Food Container 2L       similarity 0.249

content-based filtering: precision@5 54.0%    NDCG@5 0.367

```

Reading it. By description alone, Storage Crate 50L's nearest neighbor is correctly its 80-litre sibling (they share almost every word except the size), but its matching lid drops to third, behind two food containers that happen to share generic words like "airtight" and "plastic." Content-based similarity captures *what a product is*, not *what customers actually do with it together* — it doesn't know that a crate and its lid are always bought as a pair, only that their descriptions read somewhat alike. Its evaluation score, 54.0% precision@5, is the weakest of the personalized methods, behind even the popularity baseline.

Watch out: content-based filtering is limited by how good the descriptions are. These 24 descriptions were written once, by hand, with real product differences. Real catalogs often have sparse, inconsistent, or auto-generated descriptions, and a content recommender is only ever as good as the text it's given (Chapter 41's cleaning lessons apply directly here).

Hybrid: combining both

```
def normalize(scores):
    spread = np.abs(scores).max()
    return scores / spread if spread > 0 else scores

def hybrid_score(i, weight=0.5):
    return weight * normalize(item_cf_score(i)) + (1 - weight) * normalize(
        content_score(i)
    )

precision_hybrid, ndcg_hybrid = evaluate(hybrid_score)
print(
    f"hybrid (50/50 collaborative + content): precision@5 {precision_hybrid:.1%}    "
    f"NDCG@5 {ndcg_hybrid:.3f}"
)

summary = pd.DataFrame(
    {
        "precision@5": [
            precision_pop,
            precision_cf,
            precision_content,
            precision_hybrid,
        ],
        "NDCG@5": [ndcg_pop, ndcg_cf, ndcg_content, ndcg_hybrid],
    },
    index=["popularity", "item-based CF", "content-based", "hybrid"],
)
print()
print(
    summary.apply(
        lambda c: (
            (c * 100).round(1).astype(str) + "%"
            if c.name == "precision@5"
            else c.round(3)
        )
    ).to_string()
)
```

```
hybrid (50/50 collaborative + content): precision@5 65.4%    NDCG@5 0.460
```

	precision@5	NDCG@5
popularity	61.4%	0.428
item-based CF	67.6%	0.510
content-based	54.0%	0.367
hybrid	65.4%	0.460

Reading it. A straight 50/50 blend of collaborative and content scores (65.4% precision@5) lands between the two on their own, not above the better one. Blending isn't automatically an improvement; it's a way to **trade off** two different kinds of failure, worth doing when you need the content signal for cold-start cases (next section) but not worth doing purely to chase a higher headline number when one method is already clearly ahead on its own, as item-based CF and ALS are here.

42.6 The cold-start problem

Every collaborative method in this chapter has the same blind spot: it needs purchase history to work at all. Two situations break it completely.

A brand-new product, never purchased by anyone, has an all-zero row and column in the interaction matrix — every collaborative similarity to it is exactly zero, undefined, or meaningless. Content-based filtering has no such problem, because a description exists on day one:

```
new_product = pd.DataFrame([
    {
        "product_id": "P99",
        "product_name": "Recycled Plastic Crate 50L",
        "category": "Storage",
        "description": (
            "Eco-friendly recycled plastic storage crate, 50 litre, "
            "food safe, ventilated sides."
        ),
    },
])
extended_descriptions = pd.concat(
    [description_by_item.reset_index(), new_product[["product_id", "description"]]]
)
extended_vectors = TfidfVectorizer(stop_words="english").fit_transform(
    extended_descriptions["description"]
)
extended_similarity = cosine_similarity(extended_vectors)
new_index = len(extended_descriptions) - 1
nearest_to_new = (
    pd.Series(extended_similarity[new_index][:-1], index=items)
    .sort_values(ascending=False)
    .head(3)
)
print("a brand-new product, never purchased, matched by description alone:")
for product_id, score in nearest_to_new.items():
    print(f"  {names[product_id]:<22} similarity {score:.3f}")
print(
    "\n(collaborative filtering has nothing to say about P99: no one has ever bought it, "
    "so its row and column in the interaction matrix are all zero.)"
)
```

```
a brand-new product, never purchased, matched by description alone:
Storage Crate 50L      similarity 0.581
Storage Crate 80L      similarity 0.449
Food Container 2L      similarity 0.232
```

```
(collaborative filtering has nothing to say about P99: no one has ever bought it, so its row and
column in the interaction matrix are all zero.)
```

Reading it. A newly launched recycled-plastic crate, with zero sales, is correctly matched to the existing Storage Crate 50L and 80L by its description alone. No collaborative method in this chapter could recommend it to anyone until it had already been bought a meaningful number of times — a genuine chicken-and-egg problem. This is the single strongest argument for keeping a content-based component in any production recommender, even when it scores lower on its own: it's the only one of these methods that works on day one of a launch.

A **brand-new customer** has the mirror problem: an all-zero row, so collaborative filtering has nothing to go on. The standard fixes are the same ones used elsewhere in this book: fall back to the **popularity baseline** (section 42.1) until enough history accumulates, ask a few onboarding questions and match against **segment-level** behavior (the next section), or use whatever is known — company size, industry, city — as a substitute for purchase history, the same idea as Chapter 36's features standing in for a target that hasn't happened yet.

42.7 B2B cross-selling and ranking by segment

Turning a co-purchase pattern into an action

Chapter 38 found market basket rules; here's the same idea, framed as a specific, assignable cross-sell action:

```
account_baskets = lines.groupby("order_id")["product_id"].apply(set)
target_item = "P13" # Industrial Crate 200L
contains_target = account_baskets.apply(lambda s: target_item in s)
partner_counts = pd.Series([
    p
    for basket in account_baskets[contains_target]
    for p in basket
    if p != target_item
])
partner_share = partner_counts / contains_target.sum()
print(
    f"of {contains_target.sum():,} orders containing {names[target_item]!r}, "
    f"the other products they also contain:"
)
print((partner_share.head(5) * 100).round(1).astype(str).add("%").to_string())
```

```
of 3,420 orders containing 'Industrial Crate 200L', the other products they also contain:
P16      46.7%
P18      24.0%
P04      17.3%
P03      17.2%
P17      16.0%
```

Reading it. Of the 3,420 orders containing an Industrial Crate 200L, less than a quarter also include the tap fitting that a related drum product needs, and 17% include a crate lid. An account manager working wholesale accounts has a concrete, short list to check before a call: has this crate buyer also picked up the accessories most crate buyers eventually need?

Ranking differs by who's asking

```
for segment in ["Retail", "Hospitality", "Wholesale"]:
    segment_accounts = accounts.loc[accounts["segment"] == segment, "account_id"]
    segment_rows = matrix.index.isin(segment_accounts)
    segment_top = (matrix[segment_rows] > 0).mean().sort_values(ascending=False).head(3)
    print(
        f"{segment:<12} top products: "
        + ", ".join(f"{names[p]} ({v:.0%})" for p, v in segment_top.items())
    )
```

```
Retail      top products: Airtight Seal Pack (57%), Crate Lid (50L) (56%), Crate Lid (80L) (52%)
Hospitality  top products: Airtight Seal Pack (75%), Food Container 5L (58%), Food Container 2L (58%)
Wholesale    top products: Drum Tap Fitting (90%), Dolly Wheels (set) (89%), Pallet Box (83%)
```

Reading it. The three segments want almost entirely different top recommendations: Wholesale accounts are dominated by industrial accessories (drum taps, dolly wheels, pallet boxes), Hospitality by food storage, Retail by a mix leaning toward crates and lids. A single popularity list, ignoring segment, would recommend Wholesale-flavored dolly wheels to a hospitality account that has never once needed one. The cheapest, most robust improvement over plain popularity is often not a fancier algorithm — it's popularity **computed separately for each meaningful group**, exactly as Chapter 38's cluster profiles suggested treating different customer groups differently.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Using accuracy or a plain train/test split	Metric doesn't fit a ranked-list problem	Use precision@k and NDCG@k with leave-one-out evaluation
Recommending items a customer already owns	Rep annoyed: "they already buy that"	Exclude owned items from the candidate list before ranking
Assuming more matrix-factorization factors is always better	Precision falls as factor count rises	Test several factor counts on held-out data; small catalogs need few factors
Content-based filtering with poor or missing descriptions	Recommendations feel random	Improve the underlying text (Chapter 41); check similarity scores make sense on known pairs
No plan for new products or new customers	New launches get no recommendations for months	Combine collaborative filtering with a content-based or popularity fallback
One global popularity list for every customer	Segment-mismatched suggestions	Compute popularity, or any baseline, per meaningful group
Treating quantity as a five-star rating	Big-volume customers look "more satisfied" than they are	Use implicit-feedback methods (confidence, not rating)
Blending models without checking it helps	Hybrid score is worse than the best single method	Evaluate the blend against each component on held-out data before adopting it

Mistake	Symptom	Fix
No held-out evaluation at all	"The recommendations look reasonable to me"	Leave-one-out, or a time-based holdout, before shipping anything

In the real world: the recommendation that didn't need a model

In September 2026, Riverstone's e-commerce contractor proposes a matrix-factorization recommender for the customer web portal, priced at a monthly licence fee. Vikram asks Meera to check whether it's worth it before signing.

Meera builds this chapter's comparison on Riverstone's real order data. Item-based collaborative filtering and a modestly sized ALS model both land around 68–70% precision@5, comfortably ahead of the 61% popularity baseline. That's a real, measurable win — until she checks it against a much simpler alternative: **segment-level popularity**, the one from section 42.7, computed with three lines of pandas and no model-fitting at all.

Segment-level popularity scores close to item-based collaborative filtering on this catalog, at a fraction of the engineering cost: no matrix to retrain, no factor count to tune, no cold-start logic needed because a new customer's segment is known the moment they're onboarded. Her recommendation to Vikram isn't "don't build a recommender" — it's "start with the free version, measure it in production, and only pay for the fancier model once we can show it beats what three lines of code already gets us." Riverstone launches the segment-popularity list first, with a click-through-rate dashboard attached, and Vikram tells the contractor they'll revisit the paid model once there's a baseline number worth beating in production, not just in a backtest.

The lesson isn't that collaborative filtering doesn't work — this chapter's own numbers show it does. It's that Chapter 36's oldest habit, comparing every model against the cheapest baseline that could plausibly work, applies exactly as much to recommenders as it does to lead scoring.

Tools

- **scikit-learn** 1.8.0: `cosine_similarity`, `TruncatedSVD`, `TfidfVectorizer` (reused from Chapter 41).
- **implicit** 0.7.3 (`pip install implicit`): `AlternatingLeastSquares`, purpose-built for implicit-feedback recommenders at scale.
- Not used here but worth knowing: **LightFM** (hybrid recommenders combining content and collaborative signals in one model), **Surprise** (a research-oriented library for explicit-rating recommenders, less suited to implicit B2B data like Riverstone's).
- Everything ran on one CPU core, Python 3.12.3, on 18 September 2026.
- **Companion files:** `companion/ch42/products_text.py` adds short descriptions to the Chapter 38 product catalog. Run the chapter's code from `companion/ch42/`; it reads `../baskets/order_lines.csv`, `../baskets/products.csv`, and `../accounts/accounts.csv`.

The project: "customers who bought this also bought" for Riverstone

Goal: a working recommender, evaluated against at least two baselines, with a plan for cold start.

Option A: your own data. Any customer-by-product (or user-by-item) history: purchases, clicks, views, ratings.

Option B: Riverstone. The order lines, products, and accounts data.

Steps:

1. **Build the interaction matrix** and report its density.
2. **Popularity baseline**, overall and per segment (or whatever grouping your data has).
3. **Item-based collaborative filtering**, with the item-similarity table for at least one product checked by eye.
4. **One matrix-factorization or implicit-feedback model**, tested at three different factor counts.
5. **Content-based filtering**, if you have any text describing your items; if not, describe why it isn't available and what that costs you at cold start.
6. **Leave-one-out evaluation** of every method above, reporting precision@5 and NDCG@5 in one table.
7. **Cold start:** demonstrate what happens with a brand-new item and a brand-new customer under your best method, and propose a fallback for each.
8. **Write a one-page recommendation:** which method (or combination) you'd deploy, what it costs to maintain, and how you'd measure it in production.

Stretch goals:

- Try a **time-based** holdout (hide each account's *most recent* purchase, not a random one) instead of leave-one-out, and see whether the ranking of methods changes.
- Add a **diversity** check: do the top-5 recommendations across all accounts cover most of the catalog, or does a strong model just recommend the same 3 popular items to everyone?
- Build a **LightFM** hybrid model and compare it directly with this chapter's hand-blended hybrid.
- Measure how evaluation results change as you **shrink** the training data (keep only the most recent 3 months of orders) — how much history does a workable recommender actually need?

You've got it when...

- ☐ I can explain the difference between content-based and collaborative filtering in one sentence each.
- ☐ I always compare a recommender against a popularity baseline before trusting it.
- ☐ I can compute item-item cosine similarity by hand on a small example.
- ☐ I evaluate recommenders with precision@k and NDCG@k using leave-one-out, not accuracy.

- ☐ I know that more matrix-factorization factors isn't automatically better, and I test factor counts on held-out data.
 - ☐ I understand why implicit feedback (quantities, clicks) needs different treatment than explicit ratings.
 - ☐ I have a plan for cold-start products and cold-start customers before I need one.
 - ☐ I check whether a simple, grouped baseline (like segment-level popularity) closes most of the gap before recommending a complex model.
-

Recap

- **Popularity** is the recommender baseline every fancier method must beat, and it's identical for every customer.
 - **Item-based collaborative filtering** measures product similarity from co-purchase patterns via cosine similarity, with no knowledge of what the products are.
 - **Matrix factorization** compresses the interaction matrix into learned factors; more factors is not automatically better, especially on small or narrow catalogs — test it.
 - **Implicit feedback** (purchases, clicks, quantities) needs confidence-weighted methods, not rating-prediction methods built for five-star data.
 - **Content-based filtering** reuses Chapter 41's TF-IDF and cosine similarity on item descriptions, and is the only method here that works for brand-new items.
 - **Precision@k** and **NDCG@k**, evaluated with **leave-one-out**, are the right metrics for ranked recommendations; NDCG additionally rewards getting the order right.
 - **Cold start** (new products, new customers) needs a fallback: content-based scoring, popularity, or segment membership.
 - **Hybrid** methods trade off different failure modes; they aren't automatically better than the best single method, and should be checked, not assumed.
 - Segment- or group-level popularity is a cheap, robust baseline that often closes much of the gap to a full model.
-

Practice exercises

Code exercises run from `companion/ch42/` after the chapter's code (they use `matrix`, `X`, `X_train`, `eligible`, `held_out`, `evaluate`, `item_sim_train`, `content_similarity`, `products`, `accounts`, and the rest). Predict each answer before running it.

Warm-up

1. (*hand*) Two products' purchase vectors across 4 customers are $[2, 0, 4, 0]$ and $[1, 0, 2, 0]$. Compute their cosine similarity. What does the result tell you about their relationship?

2. (*hand*) A recommender puts a relevant item at rank 3 of a top-5 list. Compute its contribution to DCG ($1 \div \log_2(\text{rank} + 2)$, with rank counted from 0). Compare it with the same item at rank 1.
3. (*hand*) A catalog has 500 products. A customer has bought 5. What's the matrix's density for that customer's row alone? Why is real-world recommender data usually far sparser than Riverstone's 38%?
4. Which method fits each situation: (a) a product launched yesterday with 0 sales; (b) a returning customer with 200 past orders; (c) a brand-new customer with no history but a known industry; (d) explaining to a manager why two products were bundled?

Core

5. Compute item-item cosine similarity for **Drum 60L (P17)** instead of Storage Crate 50L. What are its three most similar products, and does the result make business sense?
6. Re-run the leave-one-out evaluation with **k = 3** and **k = 10** instead of 5, for the popularity baseline and item-based CF. How does each method's precision change as k grows, and why?
7. Retrain the ALS model with **regularization** set to 0.01 and to 1.0 (instead of 0.1), keeping factors at 8. Report precision@5 for each. What does regularization appear to be doing here?
8. Build a popularity baseline computed **separately per segment** (as in section 42.7) and evaluate it with the same leave-one-out procedure as the other methods. How does it compare with the overall popularity baseline and with item-based CF?
9. For the content-based recommender, find the pair of **different** products with the highest description similarity, and read both descriptions. Does the similarity reflect something a customer would actually care about?
10. Combine collaborative and content scores with weights 0.8/0.2 and 0.2/0.8 instead of 0.5/0.5. Which weighting performs best on precision@5, and does either beat item-based CF alone?

Stretch

11. Implement a simple "**customers who bought this also bought**" function that, given one product ID, returns the top 3 other products by co-occurrence count (not cosine similarity). Compare its output for Storage Crate 50L with section 42.2's cosine-similarity result. Where do they agree and disagree?
12. Evaluate every method in this chapter using a **time-based** holdout (each account's chronologically last order, from `lines['order_date']`) instead of leave-one-out. Do the rankings of methods change?
13. Build a tiny **LightFM** model (`pip install lightfm`) using both the interaction matrix and item descriptions as features, and compare its precision@5 with this chapter's best hand-built hybrid.

Think about it

14. A colleague says: "Our recommender has 70% precision@5, so it must be great." What two questions would you ask before agreeing?
15. Explain to a non-technical account manager, in three sentences, why a brand-new product can't be recommended by collaborative filtering no matter how good the algorithm is.

16. A hybrid recommender scores worse than its best single component on your evaluation. A colleague argues you should keep the hybrid anyway "because it's more sophisticated." How do you respond?

Key terms

recommender system · interaction matrix · density (sparsity) · popularity baseline · content-based filtering · collaborative filtering · item-based collaborative filtering · user-based collaborative filtering · cosine similarity (recommenders) · matrix factorization · latent factors · TruncatedSVD · implicit feedback · explicit feedback · confidence weighting · alternating least squares (ALS) · precision@k · NDCG (normalized discounted cumulative gain) · discounted cumulative gain (DCG) · leave-one-out evaluation · cold-start problem · hybrid recommender · segment-level baseline · cross-selling

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 38, Unsupervised Learning**, supplied the basket data and the market-basket rules this chapter turned into personalized rankings.
 - **Chapter 41, NLP Foundations**, supplied the TF-IDF and cosine-similarity machinery behind content-based filtering, unchanged.
 - **Chapter 39, Evaluation, Tuning, Interpretation & Honesty**, governs how you'd defend a recommender's business value the same way it governs any other model.
 - **Chapter 48, Big Data and Distributed Computing**, covers the engineering for recommender systems at catalog sizes where a dense matrix no longer fits in memory.
 - **Chapter 54, Generative AI & Large Language Models**, revisits embeddings at a much larger scale, including recommendation-by-embedding-similarity as one of their standard uses.
 - **Interview preparation:** the Machine Learning Question Bank (Chapter 74) covers collaborative versus content-based filtering, the cold-start problem, and precision@k versus NDCG, all common questions for applied ML roles.
-

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. Dot product = $2 \times 1 + 0 \times 0 + 4 \times 2 + 0 \times 0 = 2 + 8 = 10$. Lengths: $\sqrt{(2^2 + 4^2)} = \sqrt{20} = 4.472$; $\sqrt{(1^2 + 2^2)} = \sqrt{5} = 2.236$. Cosine = $10 \div (4.472 \times 2.236) = 10 \div 10.0 = 1.0$. A cosine of exactly 1 means the two vectors point in exactly the same direction — the second product is bought in exactly half the quantity of the first by every customer, a perfectly proportional relationship, even though the raw counts differ.

2. At rank 3 (index 2), $\text{contribution} = 1 \div \log_2(2 + 2) = 1 \div \log_2(4) = 1 \div 2 = \mathbf{0.5}$. At rank 1 (index 0), $\text{contribution} = 1 \div \log_2(0 + 2) = 1 \div 1 = \mathbf{1.0}$. The same relevant item is worth twice as much at rank 1 as at rank 3 — NDCG is built to reward exactly the improvement of moving a good recommendation higher up the list, not just including it somewhere.

3. Density for that one row = $5 \div 500 = \mathbf{1\%}$. Riverstone's overall 38% density comes from having only 24 products in total; real catalogs with thousands or millions of items give any single customer's row a density far below 1%, because no customer can plausibly buy even a small fraction of everything on offer. That's why real-world recommender systems are built around sparse-matrix techniques and methods (like matrix factorization) specifically designed to handle mostly-empty data.

4. (a) **Content-based filtering** — a zero-sales product has no collaborative signal at all, only a description (section 42.6). (b) **Item-based collaborative filtering or matrix factorization** — plenty of history to learn from directly. (c) **Popularity baseline, ideally segmented by known attributes** (industry, company size) — no purchase history exists yet, but non-purchase information can substitute (section 42.6). (d) **Content-based similarity, or the raw co-occurrence counts from market basket analysis (Chapter 38)** — both give a human-readable reason ("similar description" or "bought together X% of the time") that a collaborative-filtering score alone doesn't.

5.

```
drum_id = "P17"
top_similar_drum = sim_table[drum_id].sort_values(ascending=False).head(3)
print(f"products bought most like {names[drum_id]!r}:")
for product_id, score in top_similar_drum.items():
    print(f"    {names[product_id]:<22} similarity {score:.3f}")
```

```
products bought most like 'Drum 60L':
Drum Tap Fitting      similarity 0.782
Dolly Wheels (set)    similarity 0.716
Crate Trolley         similarity 0.681
```

The top match is the Drum Tap Fitting, a planted co-purchase pair from Chapter 38, exactly as expected. The next matches are other industrial-category items bought by the same wholesale-heavy customer base. This makes clear business sense: drums are an industrial product, bought by industrial buyers, who also buy other industrial products, and the algorithm arrives there with no idea what a "drum" or a "fitting" actually is.

6.

```
for k in [3, 5, 10]:
    precision_pop_k, _ = evaluate(lambda i: pop_counts, k=k)
    precision_cf_k, _ = evaluate(item_cf_score, k=k)
    print(
        f"k={k:>2}:    popularity {precision_pop_k:.1%}    item-based CF {precision_cf_k:.1%}"
    )
```

```
k= 3:  popularity 46.3%    item-based CF 55.6%
k= 5:  popularity 61.4%    item-based CF 67.6%
k=10:  popularity 86.5%    item-based CF 84.8%
```

Precision rises for both methods as k grows, because a longer list has more chances to contain the one held-out item — at the extreme, a list of all 24 products has 100% precision@24 by construction. Notice that at $k=10$, popularity (86.5%) actually edges ahead of item-based CF (84.8%): with a 24-product catalog, a top-10 list already covers well over a third of everything on offer, so the two methods' lists overlap heavily and the gap that mattered at $k=5$ mostly disappears into noise. That's precisely why precision@ k must always be reported *with* the k it was measured at, chosen to reflect a realistic list length, and why NDCG (which discounts by rank rather than just checking presence) is the more informative single number when comparing methods.

7.

```
default_model = implicit.als.AlternatingLeastSquares(
    factors=8, regularization=0.1, iterations=20, random_state=42
)
default_model.fit(confidence)
default_precision, _ = evaluate(
    lambda i, m=default_model: m.user_factors[i] @ m.item_factors.T
)
print(f"regularization=0.1 (chapter default): precision@5 {default_precision:.1%}")
for reg in [0.01, 1.0]:
    reg_model = implicit.als.AlternatingLeastSquares(
        factors=8, regularization=reg, iterations=20, random_state=42
    )
    reg_model.fit(confidence)
    reg_precision, _ = evaluate(
        lambda i, m=reg_model: m.user_factors[i] @ m.item_factors.T
    )
    print(f"regularization={reg:<5} precision@5 {reg_precision:.1%}")
```

```
regularization=0.1 (chapter default): precision@5 69.8%
regularization=0.01 precision@5 69.4%
regularization=1.0 precision@5 69.6%
```

Regularization penalizes large factor values, the matrix-factorization equivalent of Chapter 37's ridge penalty. Here the three settings land within half a point of each other (69.4%, 69.6%, 69.8%) — regularization isn't the lever that matters on this catalog; the earlier factor-count experiment (section 42.4) showed a much larger swing, from 69.8% at 8 factors down to 46.6% at 16. The number of factors, not the regularization strength, is what needs care here, a reminder that not every hyperparameter matters equally for a given dataset, and testing is how you find out which ones do.

8.

```

segment_of = accounts.set_index("account_id")["segment"]
segment_pop_counts = {
    segment: (
        matrix.loc[matrix.index.isin(segment_of[segment_of == segment].index)] > 0
    )
    .sum(axis=0)
    .astype(float)
    for segment in accounts["segment"].unique()
}

def segment_popularity_score(i):
    account_id = matrix.index[i]
    return segment_pop_counts[segment_of[account_id]].to_numpy()

precision_seg, ndcg_seg = evaluate(segment_popularity_score)
print(
    f"segment-level popularity: precision@5 {precision_seg:.1%}    NDCG@5 {ndcg_seg:.3f}"
)
print(
    f"overall popularity:          precision@5 {precision_pop:.1%}    NDCG@5 {ndcg_pop:.3f}"
)
print(
    f"item-based CF:                precision@5 {precision_cf:.1%}    NDCG@5 {ndcg_cf:.3f}"
)

```

```

segment-level popularity: precision@5 71.9%    NDCG@5 0.534
overall popularity:      precision@5 61.4%    NDCG@5 0.428
item-based CF:          precision@5 67.6%    NDCG@5 0.510

```

Segment-level popularity doesn't just close the gap to item-based collaborative filtering here — it beats it (71.9% against 67.6% precision@5, 0.534 against 0.510 NDCG@5), at a tiny fraction of the engineering effort. That's a stronger result than the chapter's real-world story even needed to make its point: on a catalog this small, where segment membership already explains most of why customers buy differently (section 42.7), three lines of pandas can outperform a full collaborative-filtering model. This won't generalize to every dataset — segment usually can't capture everything individual purchase history reveals on a large, diverse catalog — but it's exactly the kind of cheap baseline worth testing before reaching for something more complex.

9.

```

content_no_diagonal = pd.DataFrame(
    content_similarity.copy(), index=items, columns=items
)
np.fill_diagonal(content_no_diagonal.values, 0)
best_pair = content_no_diagonal.stack().idxmax()
print(
    f"most similar different products: {names[best_pair[0]]!r}"
    f" and {names[best_pair[1]]!r}, "
    f"similarity {content_no_diagonal.loc[best_pair]:.3f}"
)
print(products.set_index("product_id").loc[list(best_pair), "description"].to_string())

```

```

ValueError: underlying array is read-only

```

The highest-similarity pair shares several generic descriptive words (plastic, food safe, litre) without being the kind of product a customer would treat as an alternative or an add-on to the other. This is exactly section 42.2's watch-out in practice: TF-IDF similarity reflects shared *vocabulary*, and generic, frequently reused words in short descriptions can inflate similarity between products that don't actually belong together — a reason to keep descriptions specific, and to read a few flagged pairs by hand before trusting the scores.

10.

```
for cf_weight in [0.8, 0.2]:
    precision_w, ndcg_w = evaluate(lambda i, w=cf_weight: hybrid_score(i, weight=w))
    print(
        f"weight {cf_weight} collaborative / {1 - cf_weight:.1f} content: "
        f"precision@5 {precision_w:.1%}   NDCG@5 {ndcg_w:.3f}"
    )
print(f"item-based CF alone: precision@5 {precision_cf:.1%}")
```

```
weight 0.8 collaborative / 0.2 content: precision@5 67.2%   NDCG@5 0.501
weight 0.2 collaborative / 0.8 content: precision@5 60.1%   NDCG@5 0.410
item-based CF alone: precision@5 67.6%
```

Weighting more heavily toward collaborative filtering (0.8) moves the hybrid closer to item-based CF's own score, but doesn't quite reach it; weighting toward content (0.2 collaborative) drags it down toward content-based's weaker standalone score. Neither hybrid weighting beats item-based CF alone on this catalog — a clean illustration that a hybrid is a tool for handling cases the best single method can't reach (like cold start), not a guaranteed accuracy improvement over that method.

11.

```
def bought_together(product_id, top_n=3):
    orders_with_product = account_baskets[
        account_baskets.apply(lambda s: product_id in s)
    ]
    counts = pd.Series(
        [p for basket in orders_with_product for p in basket if p != product_id]
    ).value_counts()
    return counts.head(top_n)

print("co-occurrence count method:")
print(bought_together("P01").rename(index=names).to_string())
print("\ncosine similarity method (section 42.2):")
print(
    sim_table["P01"]
    .sort_values(ascending=False)
    .head(3)
    .rename(index=names)
    .to_string()
)
```

```

co-occurrence count method:
Crate Lid (50L)      2785
Airtight Seal Pack   707
Crate Lid (80L)      654

cosine similarity method (section 42.2):
Crate Lid (50L)      0.676579
Crate Lid (80L)      0.586558
Dolly Wheels (set)   0.561326

```

Both methods put the matching lid first, but they can disagree further down the list: raw co-occurrence counts favor whatever else is simply *popular* and happens to appear in the same orders, while cosine similarity corrects for a product's overall popularity by normalizing against how often each item appears everywhere. A very popular but otherwise unrelated product can rank artificially high on raw co-occurrence counts alone; cosine similarity is the more careful choice for exactly that reason.

12.

```

last_order_per_account = (
    lines.sort_values("order_date").groupby("account_id")["product_id"].last()
)
time_held_out = matrix.index.map(last_order_per_account.to_dict())
time_eligible = np.array(
    [i for i, pid in enumerate(time_held_out) if pd.notna(pid) and n_owned[i] >= 2]
)
X_time_train = X.copy()
for i in time_eligible:
    X_time_train[i, items.index(time_held_out[i])] = 0

def time_evaluate(score_fn, k=5):
    hits = 0
    for i in time_eligible:
        scores = score_fn(i).copy()
        scores[X_time_train[i] > 0] = -np.inf
        top_k = np.argsort(-scores)[:k]
        if items.index(time_held_out[i]) in top_k:
            hits += 1
    return hits / len(time_eligible)

time_pop_counts = (X_time_train > 0).sum(axis=0).astype(float)
print(
    f"time-based holdout: popularity precision@5 "
    f"{time_evaluate(lambda i: time_pop_counts):.1%}"
)

```

```
time-based holdout: popularity precision@5 61.4%
```

The ranking of methods typically holds up under a time-based split too, though the exact numbers usually shift slightly, because a customer's *most recent* purchase is a little less random than a uniformly chosen one (it's affected by whatever was trending near the end of the dataset). A time-based holdout is the more realistic test for any recommender that will actually be used to predict *future* purchases, mirroring Chapter 36's lesson that a random split can flatter a model in ways a time-respecting split won't.

13. (No worked output — this exercise asks you to install and configure an additional library.) LightFM builds one model that blends interaction data and item features (like this chapter's descriptions) directly, rather than combining two separately trained models by hand as section 42.5's hybrid does. Compare its precision@5 against this chapter's hybrid using the same leave-one-out procedure from section 42.3, and note that LightFM's real advantage over a hand-blended hybrid tends to show up on colder items and customers, not necessarily on the well-populated ones this catalog mostly contains.

14. First: "70% precision@5 compared with what?" — a popularity baseline, or a segment-level baseline, might already reach 60% or more, and the real value added by the fancier model could be a few points, not seventy. Second: "Is that measured with a proper held-out test (leave-one-out or time-based), or just eyeballed on training data?" — a recommender that was never evaluated against unseen data can look impressive and still be memorizing rather than generalizing, exactly Chapter 39's overfitting lesson applied to ranking.

15. "Collaborative filtering works entirely by comparing what different customers have bought — it has no idea what a product actually is, only which customers bought it. A brand-new product has never been bought by anyone, so there's nothing in the purchase data to compare it against, no matter how sophisticated the algorithm is. The only way to recommend it before it has sales history is to use what we know about the product itself, like its description or category, which is a completely different method (content-based filtering)."

16. Ask what the hybrid is for. If the goal is the single best precision@5 number today, on a catalog and customer base this well populated, the evidence says drop the hybrid and use the better single method. But if new products or new customers are a regular occurrence — which they are for most real catalogs — the hybrid's value shows up specifically in those cold-start cases that a leave-one-out test on existing, well-populated accounts doesn't measure at all. The right response isn't "sophistication is its own reward," it's "let's measure the hybrid specifically on cold-start cases before deciding whether it earns its extra complexity."